

Fiche de révision — Oral E6 · ColorRoom · Téo Tromprier (E2)

Format : 20 min présentation (PDF) → 20 min démonstration (app) → 20 min questions.

Objectif : connaître le projet ET le code. Tout ci-dessous est tiré du dépôt réel

NewGenesisAgency/color_room.

1. Pitch (30 secondes, à savoir par cœur)

La **ColorRoom** est un équipement scientifique du laboratoire **ENTPE / LTDS / BPMNP**, hébergé à **LUMEN – La Cité de la Lumière** (Lyon Confluence) : **2 cellules jumelles**, **42 plaques lumineuses** pilotées chacune par **32 canaux** couvrant tout le spectre visible. Le logiciel de recherche étant trop complexe pour l'initiation, j'ai développé une **application web de serious games**, embarquée sur **Raspberry Pi 5**, **100 % hors-ligne**. Ma partie (**E2**) couvre la base de données, l'API, l'interface, les jeux solo et multijoueur, la mesure colorimétrique et la documentation.

2. Les faits à connaître par cœur

Matériel (chiffres officiels — slide commune de l'équipe)

- **2 cellules jumelles** (2 salles d'analyse identiques) · **42 plaques** au total (**21 par cellule**).
- **Une plaque** : **80 cm × 80 cm** (≈ 23 cm d'épaisseur) · **2360 LED** · ≈ **300 W** rayonnés au maximum · LED disposées **en bandes sur les bords**.
- **32 canaux de commande = 32 types de LED** : **24 couleurs à spectre étroit** (du **proche UV** au **proche IR**) + **8 blancs** à différentes températures (et IR).
- **Liaison matérielle** : **bus série RS-485** (différentiel, multi-points, robuste sur longue distance) → le logiciel de supervision adresse chaque plaque / chaque canal sur ce bus.
- **Pont logiciel** : **supervision.exe** (sur le Pi) reçoit les commandes de mon API et les traduit en trames **RS-485** vers les plaques.

⚠ **À vérifier avant l'oral** : ma fiche disait avant « 18 spectrales + 14 phosphore ». La **slide officielle de l'équipe** dit **24 spectre étroit + 8 blancs = 32**. Retenir **24 + 8** (chiffre de l'équipe). Total = 32 canaux dans les deux cas.

Accès / réseau (à connaître pour la démo)

- **URL d'accès** : **http://172.17.40.39/** — depuis une **tablette** ou **n'importe quel appareil** connecté au Wi-Fi local **ColorRoom_WiFi**. (En interne le conteneur expose 3000, publié sur 8080 ; sur le réseau de la salle l'app répond à cette adresse.)
- Colorimètre CS-150/160 en **USB** sur le Pi · **Portainer** sur le port **9000**.

Pile logicielle (la mienne, E2 / équipe JavaScript)

- Next.js **16.2.1** (App Router), React **19.2.4**, TypeScript **5.5.3** (strict), better-sqlite3 **11.5.0**, Three.js **0.160.1**, Docker (multi-stage **arm64**), Portainer, Raspberry Pi **5**.

Équipe (8 — 2 sous-équipes)

- **Équipe 1 — JavaScript / React (E1 → E4)** : E1 Bonnevey (infra/Docker/CI-CD), **E2 Trompier = moi** (BDD, API, UI, jeux solo+IA+multi, mesure, doc), E3 Arbadji Ilyes (éditeur no-code + planification), E4 Akyuz (tests API).
- **Équipe 2 — Python / NiceGUI (E5 → E8)**.
- **Partenaires** : LUMEN (Cité de la Lumière, Lyon Confluence) · ENTPE / LTDS — labo **BPMNP** (Bio-ingénierie, Perception, Mécanique Numérique) · contacts Labayrade & Vella · prof Delbosc.

3. Démonstration minutée (20 min)

Prépare une **vidéo de secours** de chaque étape (au cas où le matériel/Pi bug).

1. **Connexion / inscription** (2 min) — créer un apprenant, montrer la détection de rôle, la connexion auto.
2. **Catalogue + un jeu solo** (4 min) — lancer **Color Speed** : montrer les dalles s'allumer en temps réel (3D + vraies dalles), le score.
3. **Puissance 4 contre l'IA** (4 min) — choisir un niveau (Novice → Légendaire), montrer que l'IA bloque une menace (anti-piège), parler du minimax.
4. **Multijoueur** (3 min) — Spectre Chromatique : 2 terminaux rejoignent, projection d'une couleur, scores.
5. **Mesure CS-160** (3 min) — connecter, mesurer une dalle, lire X Y Z / Lv / x,y, point sur le diagramme CIE.
6. **Tableau de bord enseignant** (2 min) — classes, scores, export CSV.
7. **Coulisses** (2 min) — Portainer (conteneurs), le `docker compose`, la base SQLite (fichier).

4. Le code à connaître (fichiers clés)

- `lib/db/index.ts` — connexion SQLite **singleton** (`dbSingleton`), `migrate()` : `PRAGMA journal_mode=WAL`, `synchronous=NORMAL`, `busy_timeout=5000`, `foreign_keys=ON` ; tables `crg_*` créées en `CREATE TABLE IF NOT EXISTS` + `ALTER` protégés ; `seedAdminFromEnv()` (compte admin via `.env`).
- `lib/auth.ts` — `hashPassword` / `verifyPassword` (**PBKDF2-HMAC-SHA512**, 100 000 itérations, sel 16 o, clé 64 o, format `sel:hash`) ; `createSession` (token `randomBytes(32)`, **30 jours**), `renewSession` (fenêtre glissante).
- `app/api/auth/register/route.ts` — inscription en **transaction atomique** (`db.transaction`).
- `app/api/auth/login/route.ts` — `verifyPassword` + cookie `crg_session` **HttpOnly + SameSite=lax** (30 j).
- `app/api/supervision/batch/route.ts` — proxy LED : **sémaphore** `HW_CONCURRENCY=2` + file d'attente (supervision.exe est quasi-série) + `AbortController` (timeout).
- `app/_components/GamePuissance4.tsx` — IA **minimax + élagage alpha-bêta**, `scoreWindow` (heuristique fenêtres de 4), profondeurs **1/2/5/9/12**, anti-piège (`winningCols`).
- `app/_components/Room3D.tsx` — vue 3D Three.js (WebGLRenderer, `forceContextLoss` au démontage, pause via Page Visibility API).
- `lib/multiplayer.ts` + `app/api/multiplayer/state/route.ts` — sessions persistées en base (`crg_mp_sessions.state_json`), lues en **polling**.

5. Fiche réponses du jury (Q&A)

Projet & cahier des charges

- « À quoi sert la ColorRoom ? » → Reproduire des éclairages à spectres précis pour la recherche sur la perception des couleurs ; mon appli la rend accessible en initiation via des jeux.
- « Pourquoi des serious games ? » → Le logiciel de recherche est trop technique ; le jeu rend la lumière/couleur compréhensible pour des apprenants.
- « Qu'as-TU fait précisément ? » → Partie E2 : BDD (7+ tables), API, UI/design, jeux solo + Puissance 4 (IA) + multijoueur, mesure CS-160, documentation. L'éditeur no-code est la partie E3.

Architecture & Next.js

- « Pourquoi Next.js ? » → Un seul projet front (React) + back (**Route Handlers**), un seul runtime Node à déployer sur le Pi ; **Server Components** pour le rendu, App Router.
- « Architecture ? » → Navigateur → routes API Next.js → couche **lib/** (auth, db, services) → SQLite + matériel (proxy supervision, pont CS-160). Tout conteneurisé.

Choix techniques (la question Node-RED)

- « Pourquoi pas Node-RED comme proposé ? » → Node-RED est **flow-based** : excellent pour automatiser des flux, mais inadapté à une vraie UI multi-pages (3D, catalogue, jeux), et ses flux JSON sont durs à versionner. React/TS/Next donnent des composants réutilisables, un **typage strict** (erreurs à la compilation), du code versionnable.
- « Pourquoi TypeScript ? » → Typage statique : un champ manquant ou une couleur mal formée est détecté par **tsc** /ESLint, pas en production.

Réseau / Docker / Raspberry Pi

- « Comment déployes-tu ? » → Image **Docker multi-stage** (deps → builder → runner) pour **arm64**, **docker compose up -d --build**, supervision par **Portainer**, app sur le port 8080.
- « Et le réseau ? » → Wi-Fi local **ColorRoom_WiFi** fourni par le Pi ; les tablettes/téléphones se connectent en local ; le colorimètre est en **USB** sur le Pi.
- « Hors-ligne, vraiment ? » → Oui : app + SQLite + matériel, tout local. (L'IA cloud Gemini est optionnelle ; un modèle local Ollama peut prendre le relais.)

Base de données / SQLite

- « Pourquoi SQLite ? » → Base **embarquée** (un simple fichier), zéro serveur à installer, idéale sur Pi ; via **better-sqlite3** (API **synchrone**, requêtes **préparées** = anti-injection).
- « SQLite tient la charge ? » → Pour quelques dizaines d'utilisateurs locaux, oui. **WAL** autorise des lectures concurrentes pendant une écriture, **busy_timeout** évite les erreurs **SQLITE_BUSY**.
- « Migrations ? » → **CREATE TABLE IF NOT EXISTS** + **ALTER** protégés : **idempotentes**, une base neuve se crée seule, une existante se met à jour sans casser.

Variables & persistance (questions « pointues »)

- **Atomique / transactionnelle** → **db.transaction()** (better-sqlite3) enveloppe **BEGIN/COMMIT/ROLLBACK** : l'inscription (user + adhésion classe) est **tout-ou-rien** (Atomicité ACID). Sinon : comptes « à moitié créés ».
- **Volatile (en mémoire)** → En JS il n'y a pas de mot-clé **volatile** ; je parle de l'**état runtime** stocké en **useRef** (ex. l'état des dalles pendant un jeu) : rapide, mais **non persistant**, perdu au rechargement — par opposition aux données **persistées** en SQLite.
- **Concurrente** → Le matériel (supervision.exe) est quasi-série ; un **sémaphore** **HW_CONCURRENCY=2** + file d'attente borne les appels simultanés.

- **Réactive** → `useState` : modifier la valeur déclenche le **re-rendu** ciblé de l'interface.
- **Environnement** → `process.env` (clé API, mot de passe admin) lue depuis `.env`, hors Git.
- **Persistante** → SQLite dans un **volume Docker** : survit aux redémarrages.

Sécurité / RGPD

- « **Comment stockes-tu les mots de passe ?** » → Jamais en clair : **PBKDF2-HMAC-SHA512**, 100 000 itérations, **sel aléatoire** 16 o, clé 64 o, format `sel:hash`. La vérification recalcule le hash et compare.
- « **Pourquoi PBKDF2 et pas bcrypt/argon2 ?** » → PBKDF2 est **natif** dans le module `crypto` de Node (aucune dépendance native à compiler sur arm64) ; 100 000 itérations donnent un coût correct. Argon2 serait plus moderne mais ajouterait une dépendance native.
- « **Les sessions ?** » → Token aléatoire (`randomBytes(32)`) en cookie **HttpOnly** (inaccessible au JS → anti-XSS) + **SameSite=lax** (anti-CSRF), **30 jours** avec **renouvellement glissant**.
- « **RGPD ?** » → Minimisation (un **pseudo** suffit, pas de nom/email) ; mots de passe hachés ; tout **local** (ENTPE) ; **droit à l'effacement** en cascade ; cloisonnement par rôle.

Jeux & temps réel

- « **Comment une dalle s'allume vite ?** » → Le navigateur ne parle pas directement au matériel : une **route proxy** relaie. Les **32 canaux** sont envoyés **en parallèle** (`Promise.all`, concurrence bornée) avec **timeout** (`AbortController`).
- « **La couleur à l'écran = la dalle ?** » → Oui, rendu **unifié** ; `CHANNEL_PROFILES` associe chaque canal à sa longueur d'onde réelle.

IA (Puissance 4)

- « **Explique ton IA.** » → **Minimax** : on simule l'arbre des coups, MAX pour l'IA / MIN pour l'adversaire ; **élagage alpha-bêta** coupe les branches inutiles ; **profondeur** limitée (1/2/5/9/12 selon le niveau). L'évaluation `scoreWindow` note chaque fenêtre de 4 cases : **défense pondérée plus fort que l'attaque** (-170 vs +130), victoire = `WIN_SCORE` (1 000 000), bonus de **centralité**. Anti-piège : l'IA ne donne pas une victoire immédiate à l'adversaire.
- « **Temps de réponse ?** » → Quasi instantané grâce à l'alpha-bêta et au poids central (exploration des bons coups en premier).

Multijoueur

- « **WebSockets ?** » → Non : l'état est **persisté en base** (`crg_mp_sessions.state_json`) et lu en **polling** de `/state`. Plus simple et robuste à déployer sur Pi, suffisant pour la cadence des jeux.
- « **Présence des joueurs ?** » → **Heartbeat** régulier ; jetons de session = **UUID** ; l'hôte (siège 1) démarre/arrête.

Mesure / colorimétrie

- « **Comment mesures-tu ?** » → Colorimètre **Konica Minolta CS-150/160** via un **pont .NET** exposé par `/api/cs160`. On lit le **tristimulus CIE XYZ**, la **luminance L_v (cd/m²)** et la **chromaticité (x, y)**, tracés sur le **diagramme CIE 1931**.
- « **ΔE ?** » → Écart de couleur entre la mesure et la cible, utilisé pour scorer la précision dans les jeux de mesure.

Qualité / tests / gestion

- « **Tests ?** » → Vérification de **types (`tsc`)** + **build de production** à chaque étape ; tests de l'API et fiches de recette côté E4 ; transactions ACID, exports UTF-8 + BOM.
- « **Gestion de projet ?** » → Git + GitHub (branches, commits clairs), développement incrémental, 15 diagrammes UML + guide technique.

UML

- « **Différence ERD / diagramme de classes ?** » → L'ERD modélise les **tables relationnelles** (clés primaires/étrangères) ; le **diagramme de classes** modélise la **conception objet** (TypeScript).

6. Questions pièges — réponses courtes

- « **C'est toi qui as tout fait ?** » → Non, projet d'équipe ; je présente **ma partie E2** (le cœur de l'appli), qui s'appuie sur l'infra de E1 et est testée par E4.
- « **Et si une dalle tombe en panne ?** » → L'API gère par plaque, l'appli continue, le jeu ne plante pas.
- « **Combien de joueurs simultanés ?** » → 1 dalle/joueur (jusqu'à 42) en local, limité par le Wi-Fi de la salle.
- « **Pourquoi pas une vraie base serveur (PostgreSQL) ?** » → Surdimensionné pour un déploiement embarqué mono-poste hors-ligne ; SQLite suffit et simplifie l'install.
- « **L'IA générative dans le projet ?** » → Hors dossier E2 ; c'est une **exploration personnelle** (génération de jeux assistée par IA locale Ollama) que je peux montrer en bonus.

7. Script slide par slide — 20 min pile (présentation PDF)

Mon deck = [ColorRoom_Presentation.pdf](#) (36 slides). Les **slides 1 → 12** sont le **tronc commun de l'équipe** (mêmes infos/chiffres/diagrammes que le deck d'Ilyes) ; à partir de la slide 13 c'est **ma partie (E2)**. Colonne « ⌚ » = durée cible ; ne jamais dépasser. Ce que je **dis** est en clair, le **mot technique à placer** est en gras.

Bloc A — Contexte & commanditaire (tronc commun) · ~5 min

#	Slide	⌚	Ce que je dis (l'essentiel)
1	Titre	0:15	« Bonjour, je suis Téo Trompier, candidat E2. Je présente le projet ColorRoom – Serious Games , réalisé pour le laboratoire LUMEN / ENTPE . »
2	Plan	0:20	« Contexte, architecture et choix techniques, puis ma contribution , et enfin la démonstration. »
3	Projet & commanditaire	0:45	Commanditaire = ENTPE / LTDS , labo BPMNP (Bio-ingénierie, Perception, Mécanique Numérique), hébergé à LUMEN – Cité de la Lumière (Lyon Confluence).
4	Installation lumineuse unique	1:00	2 cellules jumelles, 42 plaques (21/cellule). Une plaque : 80×80 cm, 2360 LED, ~300 W, 32 canaux = 24 spectres étroits (proche UV → proche IR) + 8 blancs . Pilotage des plaques par bus RS-485 .
5	Le besoin	0:40	Le logiciel de recherche est trop complexe (réservé aux chercheurs). Besoin : une interface pédagogique = ColorRoomGames , pour enseignants (créateurs) et apprenants (joueurs).
6	Acteurs & fonctions (cas d'utilisation)	0:45	Diagramme de cas d'utilisation : Enseignant → *Créer/Générer un jeu* ; Apprenant → *Jouer* ; les deux → *Mesurer (CS-160)* ; tout *inclut* *Allumer les dalles* .
7	Équipe (8)	0:35	8 étudiants, 2 sous-équipes : JavaScript/React (E1 → E4, dont moi E2) et Python/NiceGUI (E5 → E8) .

Bloc B — Architecture & choix techniques · ~4 min

#	Slide	⌚	Ce que je dis
8	Vue d'ensemble (composants)	0:45	Diagramme de composants : Navigateur → Route Handlers Next.js → couche lib/ (auth, db, services) → SQLite + matériel (proxy supervision, pont CS-160).
9	Conception orientée objet (classes)	0:40	Diagramme de classes : entités métier typées en TypeScript (User, Game, Session, Score...).
10	React/Next/TS vs JS + Node-RED	1:15	LA question clé. Node-RED est flow-based : top pour automatiser des flux, mais inadapté à une vraie UI multi-pages (3D, catalogue, jeux) et ses flux JSON sont durs à versionner . React/Next/ TS = composants réutilisables + typage strict (erreurs à la compilation) + code versionnable + un seul runtime Node à déployer.
11	Technologies mises en œuvre	0:40	Next.js 16, React 19, TS 5.5 strict, better-sqlite3 (synchrone), Three.js (3D), Docker arm64.
12	Docker · Pi 5 · déploiement	1:00	Diagramme de déploiement : Raspberry Pi 5 héberge dans Docker l'app Next.js + Ollama + supervision ; dalles via RS-485, CS-160 en USB ; clients en Wi-Fi local ; accès http://172.17.40.39/ . 100 % hors-ligne.

Bloc C — Ma partie E2 (le cœur) · ~9 min

#	Slide	⌚	Ce que je dis
13	Ma partie · interface	0:40	UI que j'ai conçue : catalogue, vue 3D temps réel (Three.js), pages jeux/mesure/gestion. Rendu unifié couleur écran = couleur dalle.
14	Données · modèle relationnel (ERD)	0:50	SQLite via better-sqlite3 ; tables <code>crg_*</code> ; migrations idempotentes (<code>CREATE TABLE IF NOT EXISTS</code>), WAL , <code>busy_timeout</code> , clés étrangères .
15	Extrait de code · données	0:50	<code>lib/db/index.ts</code> : connexion singleton , <code>PRAGMA journal_mode=WAL</code> . Dire pourquoi WAL (lectures concurrentes pendant une écriture).
16	Typologie des variables / persistance	1:00	Atomique/transactionnelle (<code>db.transaction</code> ACID) · volatile (état runtime <code>useRef</code> , perdu au reload) · persistante (SQLite + volume Docker) · concurrente (sémaphore matériel) · réactive (<code>useState</code>) · environnement (<code>process.env</code> , <code>.env</code> hors Git).
17	Ma partie · sécurité	0:50	Mots de passe jamais en clair : PBKDF2-HMAC-SHA512 , 100 000 itérations, sel 16 o ; sessions cookie HttpOnly + SameSite=lax ; RGPD (pseudo, minimisation).
18	Extrait de code · sécurité	0:45	<code>lib/auth.ts</code> → <code>hashPassword</code> (<code>pbkdf2Sync(..., 100_000, 64, 'sha512')</code>), format <code>sel:hash</code>).
19	Séquence · connexion	0:35	Diagramme de séquence : saisie → <code>verifyPassword</code> → création de session (token <code>randomBytes(32)</code> , 30 j glissants) → cookie.
20	Ma partie · gestion (tableau de bord)	0:40	Espace enseignant : classes, scores, export CSV (UTF-8 + BOM) , 3 rôles.
21	Ma partie · jeux solo	0:40	Color Speed, Simon, Tetris color-match, Puissance 4... pilotage des dalles en temps réel.
22	Séquence · exécution d'un jeu	0:35	Clic → Runtime → graphe de nœuds → <code>/api/supervision/batch</code> → dalles ; score renvoyé.
23	Diagramme d'états · jeu	0:25	Cycle de vie d'une partie (prêt → en cours → terminé).
24	Ma partie · IA (Puissance 4)	1:00	Minimax + élagage alpha-bêta , profondeurs 1/2/5/9/12 ; <code>scoreWindow</code> (défense -170 > attaque +130 , victoire 1 000 000, centralité) ; anti-piège .
25	Extrait de code · IA	0:45	<code>GamePuissance4.tsx</code> → fonction <code>minimax</code> / <code>scoreWindow</code> . Dire l'intérêt de l' alpha-bêta (couper les branches inutiles).
26	Ma partie · jeux en réseau	0:45	Multijoueur sans WebSocket : état persisté (<code>crg_mp_sessions.state_json</code>) lu en polling ; code de salon, hôte = siège 1, heartbeat .
27	Séquence · multijoueur	0:35	Hôte crée → invité saisit le code → polling de <code>/state</code> → scores synchronisés.
28	Ma partie · physique de la lumière	0:50	Mesure CS-150/160 : tristimulus CIE XYZ , luminance Lv (cd/m²) , chromaticité x,y , diagramme CIE 1931 , ΔE .
29	Séquence · mesure	0:35	Connecter → allumer dalle → mesurer → lire XYZ/Lv/x,y → point CIE.

#	Slide	⌚	Ce que je dis
30	Diagramme d'états · CS-160	0:25	Déconnecté → connecté → mesure → résultat.
31	Diagramme d'activité · remap	0:25	Transformation d'une couleur RGB vers les 32 canaux.

Bloc D — Clôture · ~2 min

#	Slide	⌚	Ce que je dis
32	Ma partie · qualité	0:40	Vérif types tsc + build prod à chaque étape ; transactions ACID ; exports UTF-8 ; tests API par E4 .
33	Démarche d'ingénieur	0:30	Besoin → conception UML (15 diagrammes) → dev incrémental (Git, ~280 commits) → CI/CD → déploiement Pi.
34	Conclusion	0:40	« J'ai livré le cœur applicatif : données, API, UI, jeux, IA, multijoueur, mesure. Solution embarquée, hors-ligne, robuste . »
35-36	Place à la démonstration	0:15	« Je vous propose maintenant de passer à la démonstration sur le matériel réel . »

Total ≈ 20:00. Si je suis en retard : raccourcir 9 (classes), 23/30/31 (diagrammes d'états/activité) — ne JAMAIS sacrifier 10 (Node-RED), 16 (variables), 24 (IA).

8. Mémo « infos complexes que je risque d'oublier »

- **RS-485** : bus série **différentiel, multi-points** (plusieurs plaques sur un même bus), insensible au bruit sur **longue distance** → c'est pour ça qu'on l'utilise pour piloter les **42 plaques**. **supervision.exe** parle RS-485 ; **mon API ne parle jamais RS-485 directement**, elle passe par ce pont.
- **URL de la démo** : <http://172.17.40.39/> (Wi-Fi **ColorRoom_WiFi**, tablette ou n'importe quel appareil). À taper en premier sur la tablette devant le jury.
- **Plaque** : 2360 LED, 300 W max, **32 canaux = 24 spectres étroits + 8 blancs, 80×80 cm**.
- **WAL** = *Write-Ahead Logging* : lectures **concurrentes** pendant une écriture (sinon **SQLITE_BUSY**).
- **PBKDF2** : 100 000 **itérations** (coût volontaire pour ralentir une attaque), **sel** unique par utilisateur, **SHA-512**, sortie 64 o.
- **Alpha-bêta** : élagage qui **coupe** les branches dont on sait qu'elles ne changeront pas le choix → l'IA explore plus **profond** en même temps.
- **Sémaphore** **HW_CONCURRENCY=2** : le matériel est **quasi-série**, je **borne** à 2 appels simultanés + **file d'attente** (sinon supervision sature).
- **Tristimulus XYZ** : 3 valeurs qui décrivent une couleur perçue (base de la **CIE 1931**) ; **x,y** = chromaticité (la couleur sans la luminosité) ; **Lv** = luminance en **cd/m²** ; **ΔE** = écart perçu entre 2 couleurs.
- **Idempotent** (migrations) : rejouer le script ne casse rien (base neuve créée seule, base existante mise à jour).

9. La vidéo de 2 min — où la placer ?

Recommandation : AU DÉBUT de la démonstration. Raisons :

1. La vraie **ColorRoom** (2 cellules, 42 plaques) est à **LUMEN**, pas dans la salle d'examen : la vidéo est **le seul moyen de montrer le système réel** exigé par le jury (« éléments réels correspondant au diagramme de déploiement »).
2. Elle **plante le décor** en 2 min (matériel, dalles qui s'allument, salle) → le jury comprend le contexte **avant** les manipulations live.
3. **Filet de sécurité** : si le Pi / CS-160 / Wi-Fi bug pendant le live, le jury a **déjà vu** le système fonctionner.

Enchaînement conseillé : vidéo 2 min (contexte réel) → puis **tests de recette en live** (R1 jouer/dalles, R2 mesure CS-160, R3 IA/multi) → outils/CI → code & évolution.

 À éviter : la mettre **à la fin** = risque de **manquer de temps** (20 min serré) et de finir sur du passif au lieu d'une manip live. La fin doit montrer **toi en train de piloter** le système, pas une vidéo.

Bon courage Téo — relis la section 5 deux fois, c'est 90 % des questions. Et la section 7 te donne le minutage exact.